



M256 Unit 12

UNDERGRADUATE COMPUTING

Software development with Java



User interface design

Unit **12**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Copyright © 2007 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1560 6

CONTENTS

1	Introduction	5
1.1	The aims of this unit	5
1.2	Terminology and scope of this unit	6
1.3	Studying this unit	6
2	A short history of graphical user interfaces	7
2.1	Graphical user interface landmarks	7
2.2	Designing the Xerox Star: a user-centred approach	12
2.3	Connections between GUI interfaces and object-oriented ideas	16
3	Usability and user-centred design	18
3.1	Usability	18
3.2	Gathering information for a user needs analysis	23
3.3	User-centred design in software development	25
4	Heuristic evaluation	29
4.1	Design principles and heuristics	29
4.2	Heuristic evaluation	31
5	Designing the Hospital System user interface	35
5.1	Gathering information	35
5.2	Structuring the GUI	36
5.3	A screen for <i>Admit Patient</i>	36
5.4	A screen for <i>Treat Patient</i>	43
6	Summary	47
	Glossary	48
	References	50
	Acknowledgements	50
	Index	51

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In *Unit 11* you learnt about the diverse benefits of paying careful attention to human–computer interaction (HCI) when developing computer systems and you also began to learn of some factors, such as design principles, that come into play when designing user interfaces.

In this unit you will learn more about designing user interfaces. This process inevitably involves making subjective judgements, and there will generally be a wide range of equally legitimate user interface designs to meet a given set of requirements. However this does not mean that the user interface designer can be as vague and unaccountable about design decisions as the popular caricature of a fashion designer. Rather, an essential element of user interface design is to be able to make explicit and articulate arguments for each feature of a proposed design, appealing principally to specific user-centred issues, as opposed to mere personal preference. This kind of accountability helps user interface designers to work with each other, with other developers and with users to create usable systems.

Fortunately, as this unit will reveal, the basics of user interface design following such self-accountability are not hard to learn. User interface design is, however, a vast area, and so our aim is to give you some insight into the issues involved rather than to impart a detailed knowledge of the subject. In particular, we do not prescribe any particular method of user interface design, as this is beyond the scope of this course; rather, we show how certain techniques generate information that should be used to guide the design process. This will enable you to start designing your own user interfaces, and to make significant improvements to existing user interfaces.

This unit begins by building on *Unit 11* in considering some historical landmarks in user interface design that illustrate various fundamental HCI issues particularly clearly. Not only does this historical discussion provide a context for the subsequent teaching, but it enables us to:

- ▶ introduce some key additional ideas that guide the design of user interfaces;
- ▶ explore the connections between graphical user interfaces and object-oriented ideas.

1.1 The aims of this unit

This unit aims to:

- ▶ analyse some historical landmarks in user interface design (Section 2);
- ▶ discuss how the concepts of usability and user-centred design translate into practical activities as part of a software development project (Section 3);
- ▶ introduce design heuristics and the technique of heuristic evaluation (Section 4);
- ▶ apply heuristic evaluation to designing parts of the user interface for the Hospital System (Section 5).

1.2 Terminology and scope of this unit

Many parts of this unit apply to user interfaces (which we may refer to simply as interfaces) in general, that is, user interfaces to all kinds of interactive products. But there is also a focus on graphical user interfaces (GUIs, or GUI interfaces) to computer systems in particular.

In this unit you will learn how the activities of designing a user interface can fit within the context of the development of a system as a whole. The term **user interface designer** is commonly used to refer to the software developer whose particular responsibility is the user interface (though on a small project this person may also be involved in developing other parts of the system). Of course, creating a user interface is inextricably linked with creating the rest of the system – the *core system* as we call it – since the purpose of the user interface is to enable a user to interact with the core system. The unit will touch on the interaction between the people involved in creating these two distinct, but related, aspects of the system, using the term *developer* to cover any one of these people.

1.3 Studying this unit

This unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on. You should note that, for some exercises and SAQs, we provide just one of a number of possible answers.

Section 5, which is largely exercise based, should require approximately half the time required for the entire unit, while Sections 2, 3 and 4 should each require roughly the same study time.

There is no computer-based work specifically associated with this unit. However, when studying this unit, you may find it helpful to refer to the computer as a useful source of ideas and illustrations. Indeed, you may often find it useful to draw on your own experience of being a computer user. This can be an extremely valuable source of ideas about user interface design; in fact the ability to put yourself in a user's place is essential.

2

A short history of graphical user interfaces

In *Unit 11* we gave a very brief review of the historical development of user interface styles. In this section we continue with this historical perspective by considering some landmarks in GUI design, and through them exploring the connections between HCI and object-oriented ideas.

While the systems discussed date back to the beginnings of interactive computer systems, they offer particularly clear insights into many of the fundamentals of good HCI, which will be useful when you begin designing your own user interfaces.

2.1 Graphical user interface landmarks

The first computer system to make a substantial contribution to the discipline of HCI is generally considered to be Ivan Sutherland's 1962 **Sketchpad** system, implemented as part of his PhD thesis.

Sketchpad

Sketchpad (see Figure 1) can be viewed as a computer-aided design system in which engineering design drawings could be created and modified by drawing with a light pen on a computer display. Several 'firsts' are claimed for this system:

- ▶ the first highly interactive graphical user interface;
- ▶ the first system to use windows of any kind;
- ▶ the first object-oriented system.

The claim that Sketchpad was an object-oriented system is based on the fact that it had templates – which were similar to classes – and from these, object-like structures were defined, with operations shared between structures having the same template. Also, the templates were organised in a kind of inheritance hierarchy. Of particular interest was Sketchpad's behaviour in interaction with the user. A key feature of the system was that if the template for, say, a bolt was altered by drawing on the screen, all displayed instances of bolts that derived from this would be updated instantly on the screen. This was particularly impressive bearing in mind that this was one of the first systems with a graphical user interface of any sort.



Figure 1 Sketchpad

SAQ 1

To which object-oriented concept do the bolts on a Sketchpad screen relate most closely?

ANSWER.....

The bolts on the screen are analogous to instances of a class, i.e. objects.

Sketchpad was created before HCI existed as a distinct scientific discipline or area of professional concern. Indeed, Sutherland's work acted as a key inspiration in HCI from its inception. This work was years ahead of its time, and many of the ideas have only recently become mainstream practice, or are still research topics. Where did the ideas behind Sketchpad come from? There are two factors that appear to have had a significant effect.

Firstly, in terms of immediate inspiration for the graphical aspects of the system, the computer system on which Sketchpad was developed, Whirlwind, belonged to the US army. Whirlwind was designed and built not long after the Second World War, by engineers with direct experience of radar as a powerful tool for quickly gathering, coordinating and displaying otherwise inaccessible information. Thus, one important set of inspirations for the interactive aspects of Whirlwind and Sketchpad appears to have been the technology of radar and the urgency of wartime information gathering and display.

Secondly, a set of more general influences can be traced to research findings in cognitive and evolutionary psychology. Early in our evolutionary history humans seem to have developed skills of inference (drawing conclusions by reasoning from evidence) for dealing with certain kinds of situation which potentially threaten survival. For example, when we see sudden movement or hear an unexpected sound, we are predisposed to look for some person or animal that caused it. The term **animate agent** is used to refer to something that can be an independently acting cause of change. Agents are generally a particularly important part of the human environment, and include predators, mates, domesticated animals, prey animals, rivals, enemies,

imagined evil spirits, family members, colleagues and friends. Crucially, animate agents generally behave differently from inanimate objects, and humans appear to have evolved special inference mechanisms for reasoning about them – mechanisms which seem to have developed from innate ways of reasoning about animals. In particular:

- ▶ If we observe that just one animal of a particular unfamiliar type (let's call it a tribble) has a certain kind of behaviour (e.g. bites, flies, lays eggs), then it is a fairly good assumption that any other tribble will do the same. This kind of reasoning is referred to as **reasoning by class**.
- ▶ If we are given the single piece of information that one type of animal is a special kind of another animal (e.g. a tribble is a special kind of bird), then any time we see a tribble, we will know that it can do everything that birds do, and probably some extra things. This kind of reasoning is referred to as **reasoning by inheritance**.

However appropriate such kinds of reasoning might or might not be, the key point is that they do not need to be taught: psychologists have found that in many circumstances people automatically apply them. By representing notions of class and inheritance explicitly, Sketchpad allowed both its programmers and its users to implicitly apply both of the above innate and powerful forms of inference.

The second seminal system we will look at, the **Xerox Star**, has in many ways had the deepest influence on the connection between object-oriented technology and GUI interfaces.

Xerox Star

In the late 1960s the Xerox Corporation, whose main income then came from photocopiers, was concerned about the coming of the 'paperless office', which was much predicted at the time. Fearful that this development might wreak havoc with the photocopier market that they dominated, Xerox decided to invent the paperless office themselves, so that they might be better placed to exploit it. Hence Xerox set up the Xerox Palo Alto Research Centre (Xerox PARC). They recruited some of the brightest computer researchers in the world, and gave them a ten-year mission to invent the office computers of the future.

A key part of the vision of the founding members of Xerox PARC's Learning Research Group, mocked up initially in cardboard, was the 'Dynabook' or 'Kiddicomp'. This was a battery-powered computer no larger than a small book (a forerunner of the laptop computer), with a colour screen, animated film-quality display, music and sound capability, massive internal storage, and network links to other machines, potentially wirelessly, as Figure 2 illustrates. The central feature of this vision was that the Dynabook should be capable of being operated, and even programmed, effectively and easily by young children: a key step, it was believed, on the road to developing computers that could be used by everyone.

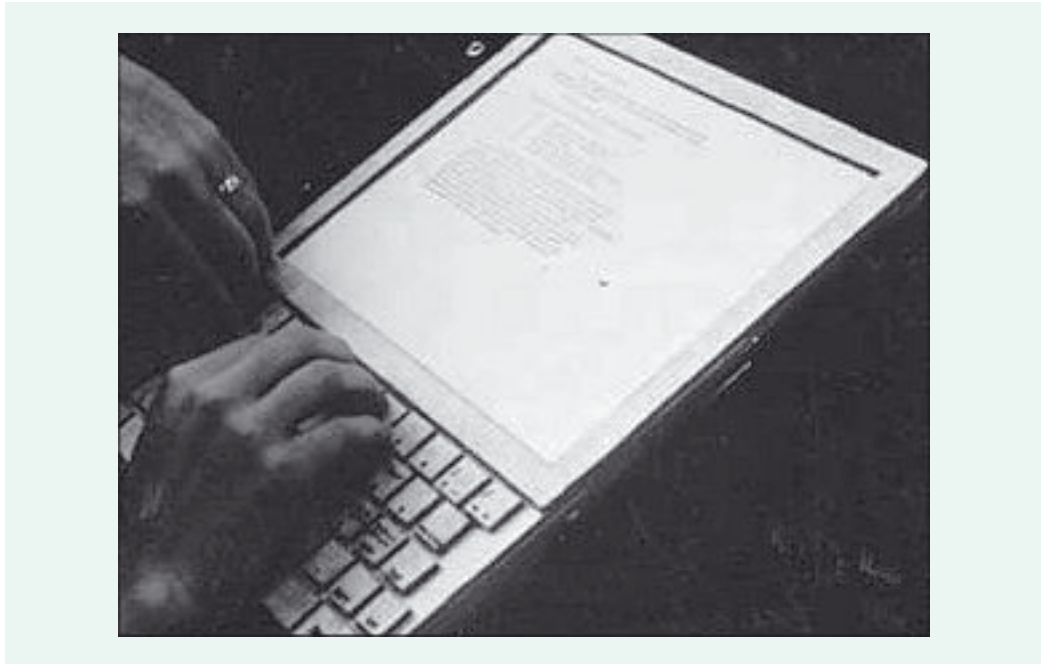


Figure 2 Dynabook mock-up

To achieve this vision the research group, led by Alan Kay, invented one of the earliest, simplest and (even by today's standards) most advanced object-oriented programming languages: Smalltalk. Dan Ingalls, one of the principal developers of Smalltalk, noted:

The purpose of the Smalltalk project is to support children of all ages in the world of information. The challenge is to identify and harness metaphors of sufficient simplicity and power to allow a single person to have access to, and creative control over, information which ranges from number and text through sounds and images.

Ingalls (1978)

The Learning Research Group was itself influenced by work undertaken in the mid-1960s on Simula, a programming language for building simulations. The designs of both Simula and Smalltalk were influenced by the innate forms of human reasoning discussed above, and the developers of both found ways to incorporate these into a programming language using ideas of objects, classes and inheritance. A key aim behind developing Smalltalk was that objects and messages should be used to implement an entire system from top to bottom, including the user interface. This could, in principle, enable the whole system to be made understandable to, and modifiable by, any user.

SAQ 2

What two innate forms of human reasoning influenced the development of both Simula and Smalltalk?

ANSWER.....

Reasoning by class and reasoning by inheritance.

Released by Xerox in 1981, the Xerox Star computer and its interface were the first major showcase for the Smalltalk project. The Xerox Star (see Figure 3) was never commercially successful (perhaps owing to its expense: a few were installed in the White House, but little used), but it was the first commercial system to incorporate key HCI concepts systematically. We will return to look at these concepts shortly.



Figure 3 The Xerox Star

The Macintosh and onwards

Steve Jobs, then chief executive of Apple Computer, and colleagues visited Xerox PARC in 1979, and were impressed by demonstrations of Smalltalk and the prototype Xerox Star. Apple went on to develop the Lisa, followed by the Macintosh, which further refined the HCI ideas developed for Smalltalk and the Xerox Star, and sold them to a mass market. The ideas that it developed were the foundation for Microsoft Windows and most other GUI interfaces since.

Thus object-oriented technology and GUI interfaces have a history that is entwined, and this is reflected in today's GUI interfaces. For example, a GUI widget can be perceived as an object that can participate in interactions with other objects within the system, entirely reflecting the object-oriented approach.

All of the above systems were developed before any significant science or practice of HCI existed, and all of them grew out of work in research centres. Later developments associated with the needs of entertainment, business and education were to prove just as influential and important to the development of HCI. In the late 1970s and early 1980s a number of commercial developments brought the personal computer (PC) vision, in several forms, to a much wider audience and helped power the development of HCI as a scientific discipline.

Commercial developments

Interactive computer games

Interactive graphical computer games began to be devised and played in the 1960s: for example Spacewar!, a 'shoot 'em down' spaceship game that first appeared in research laboratories. In the late 1970s, cheap microprocessors led to the appearance of computer games for the first time in the mass market. These games included PONG, a joystick-controlled game with a rudimentary representation of bats and a bouncing ball, and the then ubiquitous Space Invaders.

At first this development, emerging from the traditions of the amusement arcade, had little effect on mainstream computer interface design. However, this changed in the late 1980s, when former games programmers began to have a big effect on the user interfaces of commercial systems and on new commercial developments such as virtual reality systems. Some of the best examples of HCI widely available today can be found in computer games such as those on the Sony PlayStation, and the Microsoft Xbox.

The spreadsheet

In 1979 software designers Dan Bricklin and Bob Frankston developed Visicalc, the first spreadsheet application. The key idea of the interface was to make interactive the already well-understood mental model of a table of numbers. Visicalc demonstrated that, given an appropriate user interface, ordinary users could in effect rapidly develop their own programs (albeit in certain tightly prescribed areas). At the time, this was a revelation and many personal computers were bought solely to run this application.

2.2 Designing the Xerox Star: a user-centred approach

The Xerox Star was the first commercial system in which developers focused on what users found easy and what they found difficult, instead of what was straightforward to design and implement from an engineering perspective; that is, they took a **user-centred approach**. They observed users of the prototype software and classified those types of activity which users found easy to do and those they found difficult to do. These activities are shown in the table below and they are described further underneath.

Table 1 Types of activity

Easy to do	Difficult to do
concrete	abstract
visible	invisible
copy	create
recognise	generate
choose	fill in
edit	program
interactive	passive

Let us consider these contrasting types of activity.

- ▶ Most people find the concrete and visible easier to deal with than the abstract and invisible. Being faced with a blank sheet and having to create something can be terrifying, while copying something that already exists is generally much easier – as even artists such as Picasso have observed.
- ▶ Similarly, psychologists have demonstrated that recognising and choosing from a set of provided items is much easier than trying to generate them from scratch and fill in blanks. Compare the task of recognising the capital of Venezuela, say, from a list of cities versus having to remember its name.
- ▶ It is much easier to change the behaviour of a system by editing than to program a system from scratch. Consider, for example, the ease with which values can be entered into a predefined spreadsheet in comparison to the difficulty of writing code to perform a calculation.
- ▶ Finally, systems that enable interaction, i.e. those that allow the user to exert control and which give continual feedback, are generally easier to deal with than those which leave the user powerless and uninformed (passive).

Exercise 1

Give an example of an application that offers continual feedback and an example of one that does not.

Discussion.....

Here are our examples – you may well have thought of others.

- ▶ A word processor offers continual feedback on actions that affect the content and format of the text.
 - ▶ Web browsers may take a long time when trying to load a page, and give no useful feedback. In some circumstances, despite a spinning cursor for example, the user may not be able to tell whether the page is merely loading slowly or whether some error has prevented the page from being displayed.
-

From user observations such as those described above, a small set of design ideas were abstracted and used consistently in the design of the Xerox Star. These ideas included the following:

- 1 consistent use of a user's familiar mental model;
- 2 seeing and pointing rather than remembering and typing;
- 3 what you see is what you get (WYSIWYG);
- 4 universal commands;
- 5 feedback.

These ideas have become widely respected in guiding the design of user interfaces. Some, like feedback, have even evolved into design principles (see *Unit 11*, Subsection 4.1). Leaving aside feedback, which was discussed in *Unit 11*, we will look at each of these ideas in turn.

Consistent use of a user's familiar mental model

In a drastic departure from previous computer systems, users of the Xerox Star were not required to memorise an arbitrary set of commands structured in an arbitrary way by a programmer. Instead, the interface was designed so that users could use metaphors based on familiar mental models to guide them in how to use the system.

So, for example, users did not have to memorise and type in commands for printing, copying and deleting documents, sending mail and so forth. Instead, a graphical representation of a document on screen could be 'picked up' using the mouse and deposited in a graphical representation of a printer, photocopier, wastebasket or mailbox. Versions of this desktop metaphor are now more or less universal in operating systems, as you learnt in *Unit 11*, Subsection 4.3.

One of the most pervasive mental models, exploited in the Xerox Star as a metaphor, is the notion of physical space, whereby the computer screen may be considered a two- or in some cases a three-dimensional space. This ultimately gave rise to the concept of direct manipulation, which was also discussed in *Unit 11*, Subsection 4.3.

SAQ 3

What is meant by direct manipulation?

ANSWER.....

Direct manipulation is a style of user interaction based on the metaphor of physical manipulation.

Seeing and pointing rather than remembering and typing

The Xerox Star was the first commercial system not to force users to remember and type in information such as command names and file names. Instead, users could select commands, file names etc. from lists and menus. This drastically reduced mistakes caused by lapses of human memory and typing errors.

What you see is what you get (WYSIWYG)

Before the Xerox Star, most text-formatting systems employed text editors that used complex visible formatting commands embedded in the text. Thus, a document in preparation would be heavily marked up on screen (Figure 4) and would look quite unlike the final printed document (Figure 5). The Xerox Star pioneered formatting tools in which what the user saw (on screen) was what they got (from the printer), as with modern word-processing applications. WYSIWYG has not, however, made mark-up obsolete in all text-formatting systems: LaTeX, for example, which is the subject of Figures 4 and 5, is widely used for formatting technical and scientific documents.



Figure 4 Some text being formatted using LaTeX, a specialised text-formatting system

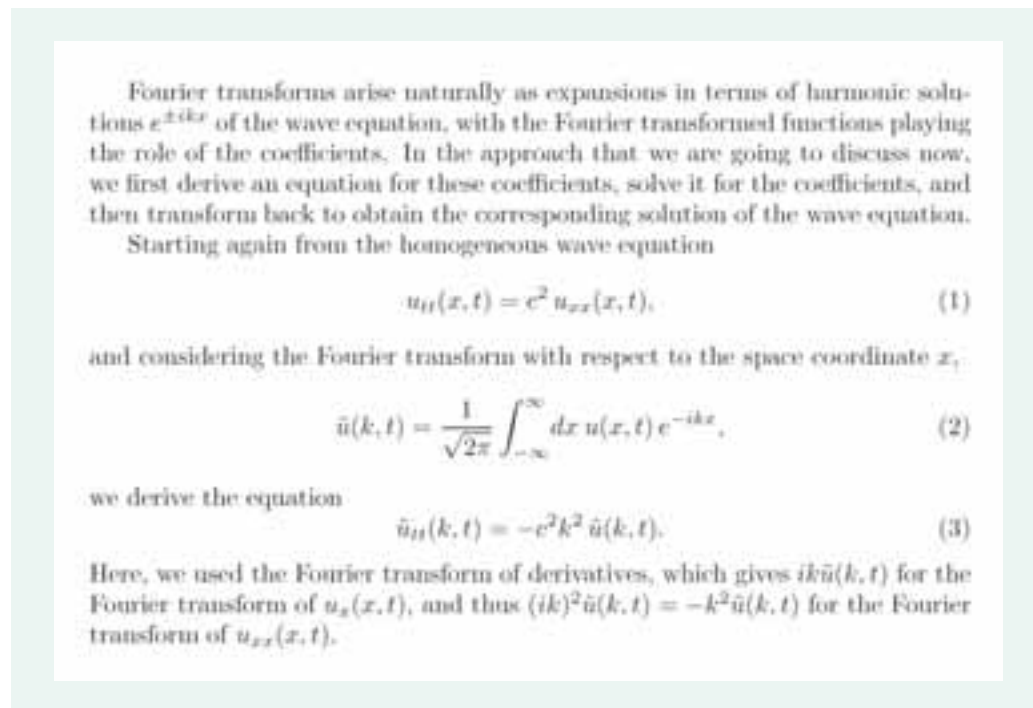


Figure 5 The formatted LaTeX document

Universal commands

In most applications and operating systems prior to the Xerox Star, the commands used to delete a file, a character, a word, a line of text, a paragraph, a picture or some other object would generally all be different. This imposed a heavy load on the memory of the user. The Xerox Star pioneered the application of a single universal command (e.g. **DeLeTe**) in all parts of the system for all kinds of 'object'.

So, for example, in today's Mac OS and Windows machines, a visible object of more or less any kind, in more or less any application, including music editors, spreadsheets, databases and statistics applications, can be deleted by first selecting the object, and then pressing a **DeLeTe** button. This kind of consistency makes it much easier to learn to use new systems.

SAQ 4

Give another example of consistency in user interfaces.

ANSWER.....

There are many such examples. We thought of the following:

- ▶ closing windows using the close button ;
 - ▶ double-clicking to open a file or folder;
 - ▶ right-clicking to bring up a menu.
-

2.3 Connections between GUI interfaces and object-oriented ideas

We will now explore a little further the natural link between GUI interfaces and the object-oriented approach to programming. The Xerox Star provides a useful case study illustrating this link.

SAQ 5

What key object-oriented concepts are connected with the origins of GUI interfaces?

ANSWER.....

The concepts of object, class and inheritance.

HCI and object-oriented technology have yet other deep connections, which the Xerox Star exemplifies as follows.

Polymorphism

Both the programming concept of polymorphism and the HCI concept of universal commands relate to the idea that the way things are modelled in the system should reflect, in an informal sense, the way that people model things. To understand this, it is necessary to recall what are meant by polymorphism and universal commands.

Polymorphism refers to the concept that the same message can be understood by more than one class of object, so that objects of different classes may respond differently but appropriately to messages with exactly the same name.

SAQ 6

Describe the HCI concept of universal commands, as exemplified by the Xerox Star.

ANSWER.....

The idea is that a small number of universal commands (e.g. **Delete**, **Copy**, **Cut**, **Paste**, **Print**) are applicable in all parts of the system for different 'objects', although each object may react in its own way.

Polymorphism relates to, and indeed supports, the concept of universal commands because it enables a message such as `deleteYourself` (sent when the user presses a **Delete** button) to be understood by all manner of objects, each of which will respond accordingly. The details of the deletion will vary with the class of object (for example, different operations are involved in the deletion of a file and the deletion of a rectangle drawn using a graphics application), but the command (pressing a **Delete** button) is the same.

Inheritance

The HCI concept of consistent use of a user's familiar mental model requires that wherever similar facilities occur in different parts of a user interface, they should work in the same way. Inheritance can help to achieve this consistency by enabling common behaviour and functionality to be shared. For example, if a type of dialogue box is defined to be a special kind of window, such dialogue boxes will inherit behaviour such as resizing and closing.

Objects

A fundamental HCI idea introduced in the Xerox Star was that of using familiar mental models and metaphors based on these mental models. As already noted, one of the most pervasive mental models, exploited in the Xerox Star as a metaphor, is the notion of physical space and physical manipulation, which ultimately gave rise to the HCI concept of direct manipulation.

These ideas can be put to work particularly successfully in object-oriented systems in which the physical space metaphor can be conveniently and consistently applied by having objects on screen correspond to software objects, with manipulation of the screen objects prompting messages to be sent to the software objects.

In this section you have discovered that not only are the histories of HCI and object-oriented programming connected, but that fundamental concepts of HCI are related to object-oriented ideas.

3

Usability and user-centred design

In *Unit 11*, Subsection 3.1, you were introduced to the important concepts of *user-centred design* and *usability*.

In this section we will outline approaches to user interface design that can be described as user centred, in that they aim to produce user interfaces having high usability.

The section will also explore the concepts of usability and user-centred design in more depth, and you will discover that, despite the apparent simplicity of these ideas, their implications are far reaching, especially when embraced systematically in a development project.

3.1 Usability

Several different aspects of user interfaces can be important, including *fun* (in the games market) and *aesthetics* (broadly, offering artistic pleasure). However, by far the most consistently important property of user interfaces is usability.

In *Unit 11* usability was described as referring to how easily and quickly a product enables users to achieve their goals, how quickly users learn to use the product's interface and what the users' attitude is towards the interface.

However, for the purposes of this unit, we will work with the following ISO definition, which as you will see is similar in content but more formally expressed.

According to ISO 9241 (Part 11), **usability** is:

The extent to which a product can be used by specified users to achieve specified goals [tasks] with effectiveness, efficiency, and satisfaction in a specified context of use.

This definition is offered by ISO in the specific context of office work with computer terminals. However, although some ISO definitions have only a limited impact, HCI professionals generally consider this ISO definition to be sufficiently useful that it is applied widely to user interfaces of diverse kinds.

It is often the case that a system and its user interface (typically the only readily perceivable part of the system) become the same thing in users' minds. Consequently, the term *usability* can sensibly be applied both to user interfaces and to the systems which they control. Though we will at times talk about the usability of entire systems, the main emphasis of this section will be on the usability specifically of user interfaces.

As you learnt in *Unit 11*, central to user-centred design is the process of *user needs analysis*, which in summary is the process of finding out about the people who will be using the system. It is in practice structured around finding answers to the key central questions stemming from the ISO definition.

- ▶ Who are the users?
- ▶ What are the tasks?
- ▶ What is the situation?

ISO is the International Organization for Standardization, the body which coordinates national standards organisations in over 150 countries.

HCI expert Don Norman advises user interface designers: 'Don't think technology: think person, task, situation'.

Who are the users?

It can be highly misleading to talk about the usability of a system without specifying the range of people who will be the users. A system that is largely unusable by the general public might nevertheless have excellent usability in its intended niche of use – for example, a computer-aided design application intended specifically for use by trained engineers. As another example, a mobile telephone interface with good usability for its young designers might be unusable in bright sunlight by older people or by those without good visual acuity.

Many systems have different users who take on different roles. For example, an online shopping system might be used both by customers for browsing products and creating orders, and by staff for administration purposes such as updating product details. For some systems there are 'indirect' users, for example, a customer on behalf of whom an operator directly interacts with a system. Identifying the users involves differentiating such roles. Each set of users may have different characteristics, which may have an impact on the prospective interface design.

So, a designer should determine who will use a user interface, and whether and in what way their characteristics are relevant to its design. The following is a sample of potentially relevant questions:

- ▶ Will the users be experts or untrained in the business area the system deals with?
- ▶ Will users be experienced computer users?
- ▶ Will users be familiar with particular operating systems?
- ▶ Can previous use of any similar systems be assumed?
- ▶ Will users be regular users or occasional users?
- ▶ Might the users have particular physical or sensory disabilities?
- ▶ What range of age and cultural background should be assumed?

The range of possible questions is open-ended, and which questions are actually appropriate will vary depending on the circumstances.

You saw in *Unit 11* (Stephen Hawking's Equaliser system, for example) that specially designed or adapted systems can be hugely empowering for people with particular disabilities. In fact, developers are increasingly making explicit efforts to include as many potential users as possible when designing systems for use by the general public.

This can be done by paying attention to **universal access** (sometimes referred to as **design for all**). The key idea is that when developing any kind of system, effort should be expended to find simple ways in which small changes could make the system more widely accessible. Indeed, to a large extent this is now required by law in many countries.

For example, operating systems often contain inbuilt accessibility features that can be applied to any application run under it. These include:

- ▶ a screen magnifier;
- ▶ the ability for text to be spoken;
- ▶ the ability for sound alerts to be changed into screen flashes.

It might be thought that manufacturers focused on making a profit would have little motivation to put effort into universal access, but in fact it has been repeatedly discovered that features put in for reasons of universal access are popular with an unexpectedly wide range of users. For example, many fully sighted users frequently use the magnifying feature. Noisy, dark and glaring environments present situations where no one may be able to operate systems adequately using 'normal' settings, therefore accessibility features potentially offer added value to all users.

What are the tasks?

However much designers know about the characteristics of a system's users, it makes little sense to try to design a usable interface without considering what the users are trying to achieve – in other words, what tasks they want to perform with the system.

The requirements document is obviously a key source of information about the tasks involved. Remember, though, that however desirable it may be to have a complete set of requirements at the outset, in reality requirements often evolve as development progresses. Throughout such a development process (to which we will return in Subsection 3.3), user interface designers work alongside other developers and users in eliciting and refining the tasks involved.

Exercise 2

Consider the following challenge, formulated by HCI pioneer Bill Buxton.

You have a choice of two drawing devices, both of which are children's toys: the Skedoodle, shown in Figure 6, and the Etchasketch, shown in Figure 7. Both allow you to draw on a screen. In the case of the Skedoodle, the movement of the drawing point is controlled by a joystick. In the case of the Etchasketch, it is controlled by two knobs. One knob moves the drawing point along the horizontal axis, and the other along the vertical axis. To draw a curve with the Etchasketch, you have to turn the two knobs at once in the appropriate directions at the appropriate relative speeds.

Buxton's challenge is to answer the question, 'Which is the best device for drawing?'



Figure 6 The Skedoodle



Figure 7 The Etchasketch

Discussion.....

The point of the challenge is to demonstrate that there is no such thing as the best interface: it all depends on the task in question. If you chose the Skedoodle, your choice can be shown to be questionable by asking you to draw a square. If you chose the Etchasketch, the same can be done by asking you to draw a circle instead.

Buxton's challenge in Exercise 2 is a graphic demonstration of one of the core truths of HCI: you cannot expect to solve the problem of finding the 'best' interface once and for all. There is no best interface. It all depends on the task in hand. As this example shows, even seemingly similar tasks (drawing a square and drawing a circle) may benefit from quite different approaches. A user interface cannot be considered in isolation from the task to be performed: the two must be considered together.

Unit 11 mentioned that a socio-technical system is comprised of both technical and human elements. Thus a person using a computer constitutes a socio-technical system. This term is important because it emphasises the need to understand not only the technical element but also the human element and the relationship between the two. In considering the tasks required, socio-technical issues such as the following need to be addressed.

- *Task allocation* How should the tasks be divided between the user and the computer? Objects and people have particular abilities. Which tasks would be appropriate for each?
- *Dialogue control* How much responsibility should the user/computer take? The user and computer interact in some way, but who takes responsibility for the interaction? In an automated factory, the computer assumes most of the responsibility for the routine operation of the factory. By contrast, with a bank ATM, the customer takes complete control (except when money, or some other service, is refused!). Thus the nature of the relationship between the user and computer needs to be clarified.
- *Collaboration* Who is the user collaborating with? In addition to collaborating with the computer system, they may be collaborating with a wide network of other people. This introduces organisational issues, such as management structure and the delegation of tasks.

What is the situation?

Even once the designers have identified the likely range of users and the tasks they will want to carry out, they have still not exhausted the basic questions to ask when planning a user interface. Other, ostensibly simple factors to do with the situation in which the system is to be used may have a great effect on the usability of the system. For example, audible alarms may be of little value in a noisy environment; an electronic diary may be useless if it is too bulky or heavy to carry around.

Depending on the particular case, questions such as the following may be relevant.

- ▶ Will the interface have users' undivided attention? If not, then by what factors might they be distracted?
- ▶ Will the environment be noisy? Will there be good lighting? Will it be wet or windy?
- ▶ Will users be wearing special clothing, eye or ear defenders etc. that may affect what they can do, see or hear?

Easy Writer: fitting a user interface to the situation

The freelance writer and consultant Steve Roberts prefers to work on the move rather than sitting in an office. He spent several years doing most of his writing while riding a specially equipped bicycle, with several on-board computers, using a chord writer on the handlebars. In a chord writer, each character is produced by pressing several keys in combination, a bit like playing the flute. This requires initial training, but the advantage is that once learnt, the keyboard need not be looked at; each finger is poised over just one key rather than several, and typing becomes very rapid and reliable.

Over time Roberts has used an array of input and output devices to let him run his business effectively in his chosen work environment as shown in Figure 8. Here, a Mac fitted under the windscreen was operated using a helmet-operated mouse. An eye-level display fitted to the helmet gave access to one of his PCs. A speech synthesiser relayed email messages. A helmet microphone was used to issue voice commands to computers, and talk on Ham (amateur) radio and mobile telephone. The trailer included solar panels and a satellite link keeping him in contact with his office and the internet.



Figure 8 Human computer interaction on the move: Steve Roberts' computerised place of work.

3.2 Gathering information for a user needs analysis

You have learnt about the kinds of information that need to be gathered about users. But how is this information gathered?

Questionnaires

Where a system is being developed for a well-defined set of users or market segment, simple questionnaires or interviews with users or prospective users may cast valuable light. The nature of such questionnaires varies depending on the nature and purpose of the particular system, and on the group being asked the questions. Questions can be simple and precise or open-ended.

For example, in the case of the Hospital System, example questions might include the following.

- ▶ Approximately how many patients are admitted each day? How much does this vary?
- ▶ Is more than one patient ever admitted by different staff at the same time? If so, describe any problems this may cause.
- ▶ Are there ever problems dealing with missing information when admitting patients? If so, of what kind, and how are these problems currently dealt with?
- ▶ What problems tend to arise in the process of admitting a patient?
- ▶ What parts of the admissions process give most problems?
- ▶ How could the admissions process be improved?

Documentation

There is often a wide range of documentary information concerning the organisation, its employees and any existing computing systems, from which it can be possible to derive information relating to the new system. These might include some of the following:

- ▶ a manual of working procedures;
- ▶ job descriptions;
- ▶ internal reports;
- ▶ annual reports;
- ▶ publicity material;
- ▶ newspaper and magazine articles;
- ▶ websites;
- ▶ instruction manuals for any existing computer system.

A surprising amount of information may be elicited from these sources, such as the age profile, educational level, training and cultural background of the users, the size and culture of the organisation and so on. For a new venture, the client may have commissioned market research, whose results the designer should study.

For example, in the case of the Hospital System it might be possible to elicit the following.

- ▶ Users: 25% of the administrators who will use the system have been in the job less than three months, and the average length of stay is ten months (elicited from an internal report).
- ▶ Tasks: in a peak period, ten patients might be admitted in twenty minutes (taken from an internal report).
- ▶ Situation: administrators have to deal with queries from medical staff while admitting patients (elicited from job descriptions).

User observation

It is almost invariably useful for developers to observe users before work on a new system begins. Answers to many questions about the users, their tasks and their situation may be found by such apparently simple observation. For the Hospital System, if users were observed at work before the inception of the new system, the following hypothetical information might be elicited.

- ▶ Users: temporary administrators often do not know which doctors are allocated to which teams.
- ▶ Tasks: sometimes patients have to be admitted using incomplete information, which must then be added later.
- ▶ Situation: occasionally patients are noisy or violent, which could interrupt staff trying to use the system.

User needs analysis document

Once the user needs analysis has been conducted, the information that it generates about tasks, users and situations is usually documented in a **user needs analysis document** to give designers a concrete basis on which to work. This document may take different forms for different projects but, as you would expect from what has been said above, will typically cover the following three basic areas.

- ▶ *User characterisation* The characteristics of the users which might be relevant to the design of the interface.
- ▶ *Task analysis* The tasks the users will need to carry out. The task analysis is likely to refer to the use cases already detailed in the requirements documentation. It may also give background details on the tasks, descriptions of how they fit together, what wider goals they meet etc.
- ▶ *Situational analysis* A summary of the environment in which the system is being used, and any ways in which this might impact upon the use of the interface.

SAQ 7

Suggest three example questions that a *user characterisation* might answer.

ANSWER.....

Here are some examples – you may well have thought of different ones.

- ▶ Will users have specialised knowledge?
 - ▶ Will most users be frequent users or first-time users?
 - ▶ Will their eyesight be good?
-

The information in the user needs analysis document can be used as the basis for recommendations for the first prototype interface. For example, with a rapid staff turnaround a priority might be for the Hospital System user interface to be easy for users to learn to use. Dealing with ten admissions in twenty minutes means that the staff may be under considerable time pressure; thus the user interface may have to enable users to rectify particular types of error easily. Because of possible frequent distractions, it might be desirable for each screen to be self-contained.

3.3 User-centred design in software development

You can see from the preceding discussion that in a user-centred approach, interface designers need to discover a considerable amount of information about the users. You know that design concepts and principles may also inform their work. All of this takes place in the context of the development of the system as a whole. So how does user interface design fit in?

While the core system and the user interface are distinct, they are of course interdependent. They must cooperate so that the core system is capable of responding to requests from the user interface and of producing the information that the user interface needs for display. Thus, in practice, the development of the core system and the development of the user interface usually proceed in parallel, with the activities of each continually informing those of the other.

Prototyping

As you know, software development is often carried out iteratively. Development phases are repeated cyclically, with each cycle building in more and more of the system's functionality. *Unit 1* introduced the idea of a prototype, usually a central aspect of iterative development, which is an early version of the system used to explore what the system should do and how this could be accomplished. In general each cycle improves the prototype until it results in the full working system.

User interface design usually goes hand in hand with prototyping. Having determined initial information about the users, their tasks and their situation, a first prototype interface can be created. This prototype and its use by the users is analysed, and the results are fed into the next development cycle, which improves the prototype. These results may be concerned with the functionality of the system (suggesting additional tasks that the system should carry out, for example), or they may be concerned with the layout or other aspects of the interface, or they may involve a combination of the two.

There is a range of different approaches to user interface prototyping, which we will not investigate in detail in this course. At the most complicated end are fully functional software prototypes; at the simplest end are paper sketches of user interfaces.

Surprisingly useful investigations can be, and often are, carried out with simple paper sketches of user interfaces. In analysing a paper prototype, users have to imagine that they are interacting through a real interface, with the designers describing what the results of their actions would be. Simple and cheap prototypes are very useful, because they let designers explore lots of ideas at low cost. Moreover, a prototype that is very sophisticated may be psychologically difficult to abandon, even though it is poor in some respect, for reasons such as the following:

- ▶ if a prototype has been expensive to produce, the developers and/or client may not like the idea of 'writing it off';
- ▶ if users have worked a lot with a prototype, they will be familiar with it and may not welcome a change;
- ▶ if the client has had considerable involvement in the building of a prototype, they may feel a sense of commitment to it.

In Section 4 you will learn more about how relevant design concepts can guide the designs of the prototypes.

Sketching paper prototypes is an approach you will learn about later in this unit.

Usability testing

Once a prototype interface exists, **usability testing** begins to assess the usability of the interface and feed results into the next cycle so that the prototype can be improved. This can be a complex business involving special equipment and specialist staff, but it can also be carried out in very simple, quick, informal ways, which can be highly illuminating.

'Cheap' user testing

Larry Tesler, as Vice President of Advanced Products at Apple, is on record as saying:

I've been personally involved in cheap user testing for about 28 years, including my tests for the Lisa [forerunner of the Macintosh computer]. The equipment was me, a Lisa, two chairs, a notepad and a pen. The subjects were right out of employee orientation [i.e. inexperienced new employees]. The tests ran one hour. I ran 3 to 5 subjects on each version of the application. I spent another few hours analysing the notes and writing the report. Cheap. Effective.

A simple form of usability testing is to observe users, enabling the developers to get a clearer picture of the users' attitudes to the system, their personal preferences and so on, as well as noting specific details such as the time taken to achieve tasks, and the mistakes made.

Before observing users, it is necessary to decide on the precise details of the observation, as follows.

- ▶ Will the observation be formal or informal? In a formal observation the environment is tightly controlled and the tasks are prescribed. Informal observation allows for more flexibility and interaction between the developer and the user.
- ▶ Will the observer (the developer) be active or passive? Will the developer ask questions of the user or just sit and watch? Will the developer participate in the activity?
- ▶ Will the observation be direct or indirect? In direct observation, the developer sits with the users and watches them. In indirect observation, the developer records them on camera.

There are advantages and disadvantages to each of these variations. For example, with direct observation the presence of the developer may alter the behaviour of the users (they might feel self-conscious). Asking questions can be helpful to the developer's understanding of why the users are acting in a certain way, but it may distract the users and cause them to behave differently.

SAQ 8

Suggest one possible advantage and one possible disadvantage of indirect observation.

ANSWER.....

With indirect observation, the participants may be conscious of the camera. However, the recording can be viewed repeatedly, which means that more information may be captured.

Think-aloud is a simple usability testing technique which involves the user speaking aloud what they are thinking when using the user interface to perform tasks (often specified by the developer). Thus the user explains what they are trying to achieve, why they press a particular button, what they are thinking when they make an error and so on.

Usability testing findings can result in a number of possible recommendations for a user interface. For example, having noted that an administrator often needs to record the treatment of a patient as soon as they are admitted to a ward, a recommendation for the next prototype might be to have a link from the screen for admitting patients to the screen for treating patients.

Effectiveness, efficiency and satisfaction

A key aim of usability testing is to assess the *effectiveness*, *efficiency* and *satisfaction* of the prototype interface, as highlighted by the ISO definition of usability. Each of these terms can be interpreted in a wide variety of ways, and there are no universally agreed measures; rather, different measures exist that may be applicable in different situations.

Effectiveness broadly means getting the right thing done. One crude but useful way of measuring effectiveness is in terms of numerical *task success rates* or conversely *error rates*. So, for example, one might wish to assess the effectiveness of a new prototype interface for looking up train times, against the previous prototype. One simple approach would be to observe users attempting a number of tasks using the two interfaces. A rough and ready measure of the relative effectiveness of the two interfaces could be gained by counting the number of tasks successfully completed, and perhaps additionally by counting the total number of mistakes observed.

Efficiency is an indication of how much of a resource has to be expended to get the job done. It is often measured in terms of the *time taken to complete a task*.

User satisfaction in interacting with the user interface is clearly a subjective matter, but a very important one. It may depend on subtle factors such as the order in which dialogue is sequenced. For example, consider an online car-hire system that spends ten minutes asking you what kind of car you want to hire, before revealing that in any case there are no cars available at the local depot.

Users often relate satisfaction to their workload, which can be assessed and measured in a variety of ways, from using straightforward questionnaire-based surveys to using more complex tools such as the **NASA TLX (Task Load Index) tool**. This tool was developed at NASA as a way of measuring a user's impression of their workload when carrying out some task. It is based on a multidimensional rating procedure that calculates an overall workload score based on a weighted average of ratings on six subscales: frustration, mental demands, physical demands, temporal demands, own performance and effort. Though of limited use as an absolute measure, this score can be valuable in comparing the subjective workloads of different tasks.

Although effectiveness, efficiency and satisfaction are different characteristics, they are often dependent. For example, the unfortunate behaviour of the online car-hire system mentioned above would not only be damaging to user satisfaction but would also be inefficient (a better-designed system would more promptly reveal the lack of locally available cars).

Because of this dependence, measures taken to improve one characteristic may unintentionally damage others. For example, the introduction by a company of a telephone voice menu system to screen telephone calls from customers may be both efficient and effective in terms of saving employees' time, but at the cost of reduced user satisfaction.

SAQ 9

On what basis could each of the following be measured for a particular prototype? Give one example for each:

- ▶ effectiveness;
- ▶ efficiency;
- ▶ satisfaction.

ANSWER.....

- ▶ Effectiveness could be measured on the basis of task success rates.
 - ▶ Efficiency could be measured on the basis of task completion times.
 - ▶ Satisfaction could be measured using the NASA TLX tool, that is, on the basis of frustration, mental demands, physical demands, temporal demands, own performance and effort.
-

In this section you have learnt that in a user-centred approach to design, designers gather information at all stages of the development process to inform their development of a user interface that will have high usability. Drawing on the information contained in the user needs analysis document, an initial prototype user interface is created, analysed in various ways, and successively refined as part of an iterative approach to software development.

In both this unit and *Unit 11*, you have learnt that design principles and other concepts (such as the use of metaphors and universal commands) provide additional guidance relevant to designing a user interface. In the next section you will study how such guidance can be built into a practical approach to user interface design.

4

Heuristic evaluation

As you have seen, usability is an overriding consideration in designing user interfaces. So a key question is: how can user interfaces be designed to maximise usability? The first generation of user interface designers (for example, the team that created the Xerox Star) had to discover the art as they went along, but nowadays designers can benefit from the hard-won experience of these pioneers, while of course still drawing on their own intuition.

This section, therefore, looks at the following factors that can guide the user interface design process:

- ▶ ideas, principles and heuristics;
- ▶ heuristic evaluation.

4.1 Design principles and heuristics

The Xerox Star designers strove hard to distil their experience in designing the Xerox Star into the first well-known set of ideas guiding the design of user interfaces, which you learnt about earlier in this unit.

SAQ 10

State two of the design ideas that emanated from the design of the Xerox Star.

ANSWER.....

Any two of the following form an appropriate answer:

- ▶ consistent use of a user's familiar conceptual model;
- ▶ seeing and pointing rather than remembering and typing;
- ▶ WYSIWYG (what you see is what you get);
- ▶ universal commands;
- ▶ feedback.

Along with feedback, you also learnt in *Unit 11* (Subsection 4.1) about the important design principles of good visibility and affordance, which guide the design of user interfaces.

SAQ 11

What is meant by the affordance of a GUI widget?

ANSWER.....

Affordance is how clear it is to operate the widget.

When a piece of guidance is described as a principle, it means that this piece of guidance should be more or less universally applicable. Consequently, user interface design principles are very wide ranging, and typically encapsulate a lot of hard-won HCI wisdom.

Design principles can in most cases be easily applied; for example, buttons usually have a kind of inbuilt affordance in that they look 'pushable', so it is clear how to operate them. In other cases it may not be clear exactly how to go about applying a particular principle in a particular situation. For example, the principle of good visibility states that the purpose of a widget should be very clear to the user, but it gives no hint about what can actually be done to achieve this goal. In addition, there are other design ideas that are very important but not general enough to count as principles.

For such reasons many HCI experts advocate using a small number of pieces of relatively concrete advice, known as **design heuristics** (sometimes called **usability heuristics** or simply, in the context of user interface design, **heuristics**). Heuristics can be considered as rules of thumb: they may not always be applicable, but they are frequently useful. They are specific items of advice that help translate the principles and the ideas you have learnt about into practice, and they supply concrete details that general principles do not cover.

Several versions of design heuristics are widely used, such as a well-known version developed by respected HCI experts Rolf Molich and Jakob Nielsen. The version presented below is a variant of this and related methods.

- 1 *Use simple and minimal dialogue* Present concisely the required information and instructions; and only present the minimum necessary for the task at hand.
- 2 *Use simple grouping and ordering* Group and order information in a way that the user will find simple and natural.
- 3 *Speak the users' language* Use concepts and terminology that the users are familiar with.
- 4 *Minimise the users' memory load* Avoid requiring users to memorise information, instead giving them opportunities to recognise, select, confirm or modify.
- 5 *Allow for undo-ability and commitment* Allow users to change their minds and undo provisional choices without having to start again from scratch. Distinguish clearly between exploring options and committing to choices.
- 6 *Be consistent* Ensure that the appearance of the interface, and the manner in which it is used, applies consistently right across the interface.
- 7 *Provide feedback* Provide appropriate and timely feedback to the user in response to their actions.
- 8 *Include clearly marked exits* Enable users to backtrack if they make a mistake, and to leave the system when they are ready.
- 9 *Prevent errors* Predict and minimise the potential for user errors.
- 10 *Provide appropriate error messages* Provide error messages in plain language, making clear from the users' point of view what is wrong, and suggesting a constructive solution.

Note the following.

- The heuristics place the user centre stage in the design process. *Speak the users' language*, for example, cannot sensibly be applied without reference to who the users are. Thus application of these heuristics fits naturally into a user-centred design process.
- The heuristics are not completely independent. For example, minimising the amount of information users need to remember contributes to preventing errors. Conversely the application of one heuristic may militate *against* the application of another. For example, in order to minimise the amount of information users need to *remember*, it can be easy to *provide* so much information that the simplicity of the dialogue between the user and the system is diminished.

The word 'heuristic' is derived from the Greek (meaning 'I find'), from which Archimedes' famous exclamation of 'Eureka!' was derived.

- Feedback, which was covered in some detail in *Unit 11*, also features here as the subject of a design heuristic. This is not surprising, in view of the generally central role of feedback in user interfaces. You may be wondering how the associated concepts affordance and visibility, as discussed in *Unit 11*, fit in with the heuristics. Although they have wider roles, for the purposes of this course, affordance and visibility play their most useful role when choosing or critiquing individual widgets, or other small-scale features of interfaces. The design heuristics, by contrast, apply naturally at any level of detail, including both individual elements and the user interface as a whole.

4.2 Heuristic evaluation

Heuristics such as those above can be applied as part of creating and improving prototype user interfaces (or indeed, improving the user interface of an existing system) in an activity known as **heuristic evaluation**. A heuristic evaluation of a user interface is an assessment of the interface with reference to the design heuristics, by simply considering the heuristics in turn to see if any of them inspire an improvement to the interface. This is in essence very straightforward to carry out but can yield significant results.

In a real project, heuristic evaluation would complement other activities such as user needs analysis and usability testing; however, as we cannot simulate these activities here, in the examples which follow you will apply only heuristic evaluation.

Imagine that the Big Entertainment Corporation requires a new system to facilitate telephone enquiries and bookings for entertainment events held in several theatres. The system will be used by call-centre staff responding to telephone calls from the public.

Figure 9 contains three diagrams of screens (windows) taken from part of a prototype user interface for the Big Entertainment System. A diagrammatic representation of a user interface, which may show the navigation through screens as in Figure 9, is known as a **storyboard**.

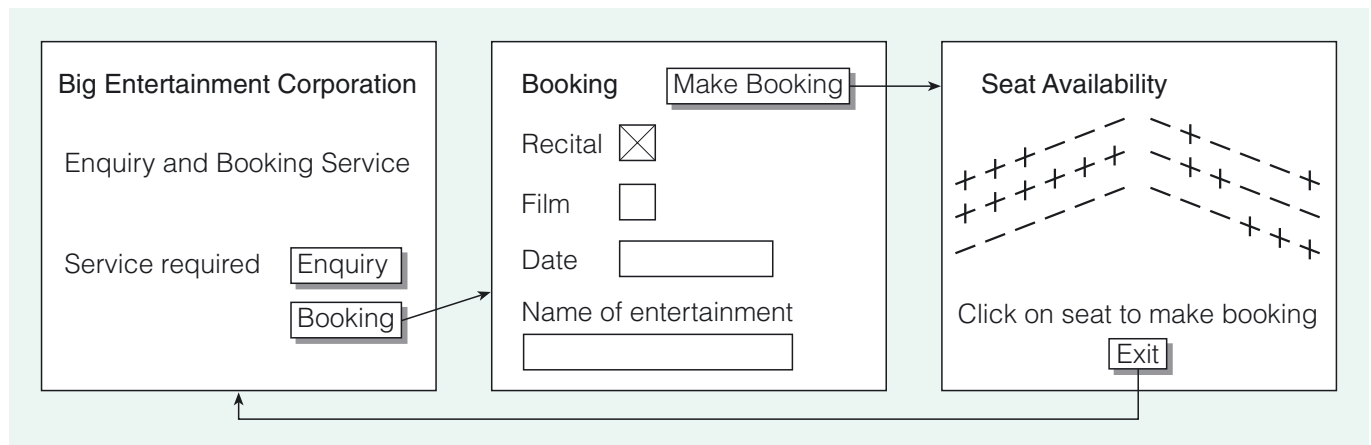


Figure 9 A prototype user interface for the Big Entertainment System

The user interface should be self-explanatory. The **Seat Availability** screen is a common type of screen in booking systems: it displays available seats as dashes and booked seats as crosses. In order to book a seat, the representation of that seat must be selected.

The user interface for the Big Entertainment System has a number of shortcomings, which can be identified by the evaluation process. We will identify some of them by considering some of the design heuristics in the list above.

- *Include clearly marked exits* Navigation is inflexible: if the user selected the **Booking** button by accident, they would need to select **Make Booking** and **Exit** to recover from the error.

- *Speak the users' language* The words 'Enquiry' and 'Booking' are rather vague: they give no information about what can be booked or what sorts of enquiry can be answered.
- *Minimise the users' memory load* The user has to remember the names of entertainment events to enter them in the **Booking** screen.

Note that different design heuristics can identify the same problem. For example, the inflexibility of navigation is indicated both by *Use simple grouping and ordering* and by *Include clearly marked exits*.

Exercise 3

By considering the following design heuristics, identify further faults with the user interface for the Big Entertainment System.

- (a) *Be consistent*
- (b) *Prevent errors*

Discussion.....

Here are some possible faults (you may have identified different ones).

- (a) *Be consistent* The action buttons are all in different places on the screens.
- (b) *Prevent errors* On the **Booking** screen, the check boxes will allow the erroneous selection of both Recital and Film.

Of course it is not much use simply to identify faults in an interface: suggestions for improvement are required. A **design rationale** documents the designer's suggested improvement, as in the following examples.

- **Design heuristic** *Include clearly marked exits*
Issue Navigation is inflexible: if the user selected the **Booking** button by accident, they would need to select **Make Booking** and **Exit** to recover from the error.
Suggested improvement Include an **Exit** button on the **Booking** screen.
- **Design heuristic** *Minimise the users' memory load*
Issue The user has to remember the names of entertainment events to enter them in the **Booking** screen.
Suggested improvement Change the **Name of entertainment** text field to a list of event names.

Exercise 4

Draw up a design rationale documenting an improvement that you would suggest to the interface in Figure 9 in respect of the fault identified in Exercise 3(b).

Discussion.....

Design heuristic *Prevent errors*

Issue On the **Booking** screen, the check boxes will allow the erroneous selection of both Recital and Film.

Suggested improvement Change each check box to a radio button, to enable only one of Recital and Film to be selected.

The following exercise gives you further practice in applying design heuristics to identify faults and suggest improvements.

Exercise 5

Part of a prototype user interface for the Neverbein Hospital System is illustrated by the storyboard in Figure 10. You can see that there are some differences between this example and that of our Hospital System. Different information about patients and their treatment is recorded; for example, information about a patient's diet and religion is required. Several age codes are used: A (0–18), B (19–25), C(26–35) etc. Several religion codes are used: C (Christian), M (Muslim), N (none) etc. We will assume that the system is used by nurses in admitting and discharging patients and in recording and viewing details about patients and their treatment.

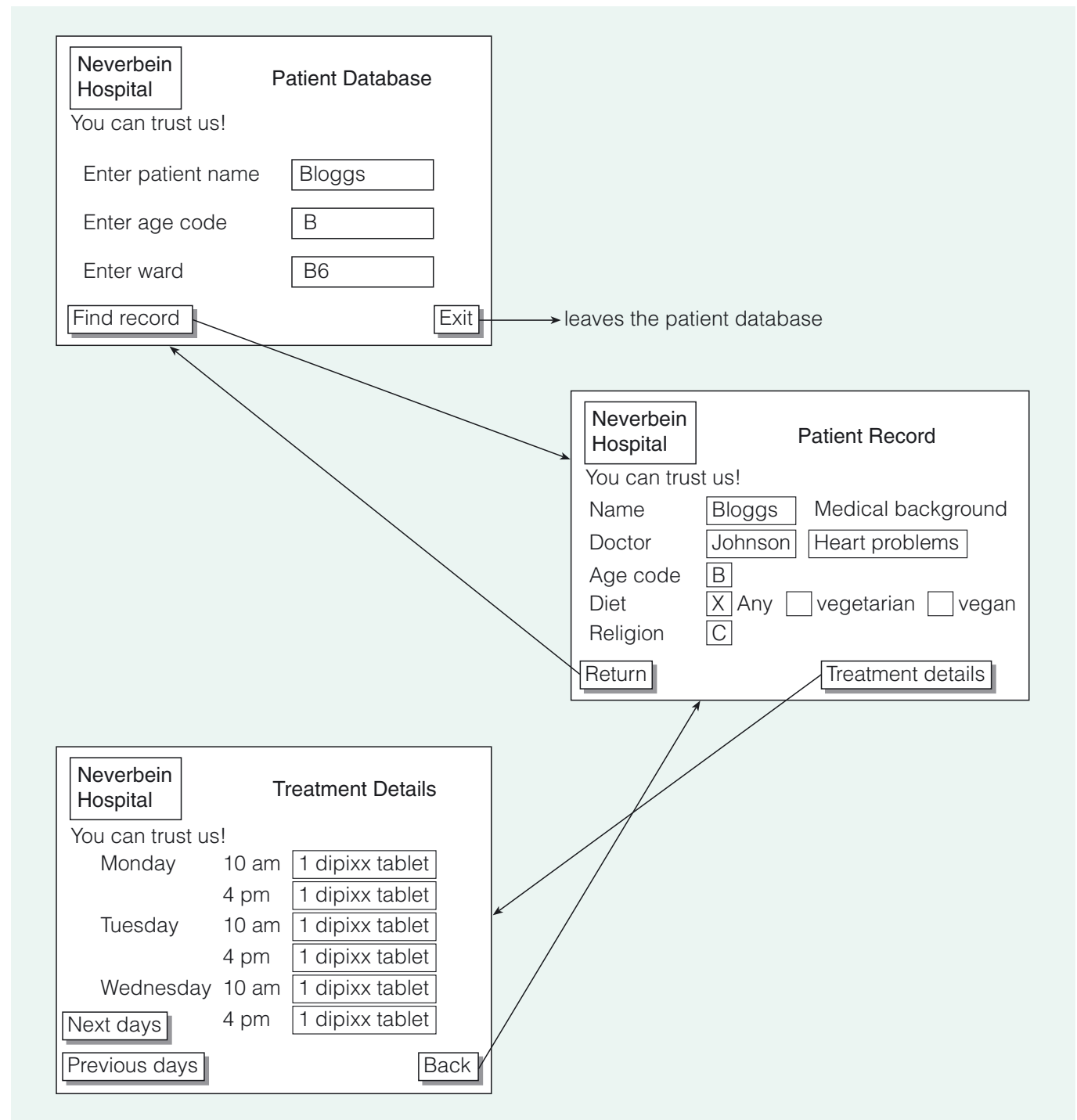


Figure 10 A prototype user interface for the Neverbein Hospital System

For each of the following design heuristics, identify a corresponding fault with the prototype user interface, and decide on a consequent improvement. Give your answer in the form of a design rationale.

- (a) *Use simple grouping and ordering*
- (b) *Speak the users' language*
- (c) *Minimise the users' memory load*
- (d) *Be consistent*

Discussion.....

Here are some sample answers. You may well have identified different faults and/or suggested different improvements.

- (a) **Design heuristic** *Use simple grouping and ordering*

Issue Medical background information is displayed separately from treatment information, but it is likely a user will want to see all this information together.

Suggested improvement Display medical background on the **Treatment Details** screen, rather than on the **Patient Record** screen.

- (b) **Design heuristic** *Speak the users' language*

Issue The term 'record' may be unfamiliar to novice users.

Suggested improvement Rename (i) the **Find record** button on the **Patient Database** screen as **Get patient details** and (ii) the **Patient Record** screen as **Patient Details**.

- (c) **Design heuristic** *Minimise the users' memory load*

Issue The **Treatment Details** screen does not remind the user which patient is being referred to. This could have serious repercussions if a nurse is interrupted while using the system.

Suggested improvement Display the patient's name on the **Treatment Details** screen.

- (d) **Design heuristic** *Be consistent*

Issue The naming of the buttons is inconsistent: the **Treatment Details** screen uses a **Back** button to return to the previous screen, while the **Patient Record** screen uses **Return**.

Suggested improvement Rename the **Return** button on the **Patient Record** screen as **Back**.

In this section you have been introduced to design heuristics, which are generally useful rules of thumb for designing a user interface. You were also introduced to a specific set of heuristics, and learnt how a given interface can be evaluated in respect of the heuristics using a process called heuristic evaluation. Finally you learnt how to document an improvement to a user interface in the context of a particular design heuristic.

In the next section you will return to *our* Hospital System and carry out heuristic evaluation as part of designing prototype interfaces for it.

5

Designing the Hospital System user interface

In this section you will participate in the design of parts of a user interface for the Hospital System.

You will learn not only how to design an initial prototype GUI interface but also how, in the spirit of iterative development, to improve it. The approach you will take is based on the design heuristics introduced in Section 4, and can be described as user-centred, although as you will not have actual users to test out your prototypes on, you will rely more heavily than usual on the design heuristics. You will also be applying ideas from *Unit 11* about choosing appropriate widgets.

To make this potentially very complex activity manageable, some simplifications are necessary.

- ▶ The Hospital System GUI interface will be structured so that navigation between screens is very simple. You will not need to consider issues of navigation as you did in Section 4.
- ▶ For each screen, you will sketch an initial prototype design and then one improved prototype. We will not take the prototyping process further than this.
- ▶ The widgets you are required to select from are relatively simple: they are those discussed in Section 5 of *Unit 11*.

5.1 Gathering information

To keep M256's introduction to software development as straightforward as possible, the requirements document for the Hospital System was made deliberately simple. It gives very little detail about the users or typical situations of use, although the basics of the tasks are spelt out in the descriptions of the use cases. This is true to life for many such documents in the early development stages. In a real project, as described in Section 3, the developers would carry out various activities to discover more information about the users, their tasks and their situation.

SAQ 12

What activities might the developers carry out to gain more information about the users, their tasks and their situation, *before* work on the system begins?

ANSWER.....

The following activities were mentioned in Section 3:

- ▶ surveying users and potential users via questionnaires;
- ▶ analysing documentation;
- ▶ user observation.

In a fully fledged user-centred design process, all of the information available about users, tasks and situation would feed into the interface design. For the purposes of this section, however, your key focus will be to sketch and improve an interface making the best use of the information that is available in the requirements document. Where necessary, we will provide you with additional information.

5.2 Structuring the GUI

The Hospital System is typical of complex systems in that several distinct tasks are required of it; these are specified as use cases in the requirements document. On the basis that the GUI's role is to enable the user to achieve those tasks, an important element in designing it is to decide how the tasks will influence the overall structure of the user interface – in particular, how the GUI will comprise different screens and how the screens may be linked for navigation purposes.

When designing a set of related screens, questions of navigation and exits can become quite complicated. Different possible routes through the screens are analysed, and various possibilities to enable navigation, such as additional buttons or menus on the screens, are assessed. One useful design tool in such situations is the storyboard, which you met in Section 4.

Such analysis can be a complex business, so for the Hospital System we will simplify things considerably by having a separate screen for each use case. The screens will be combined in a particularly simple fashion as a set of **tabbed screens**, giving the outline structure illustrated in Figure 11. To move from one screen to another, all that is required is to click on the labelled tab of the desired screen.

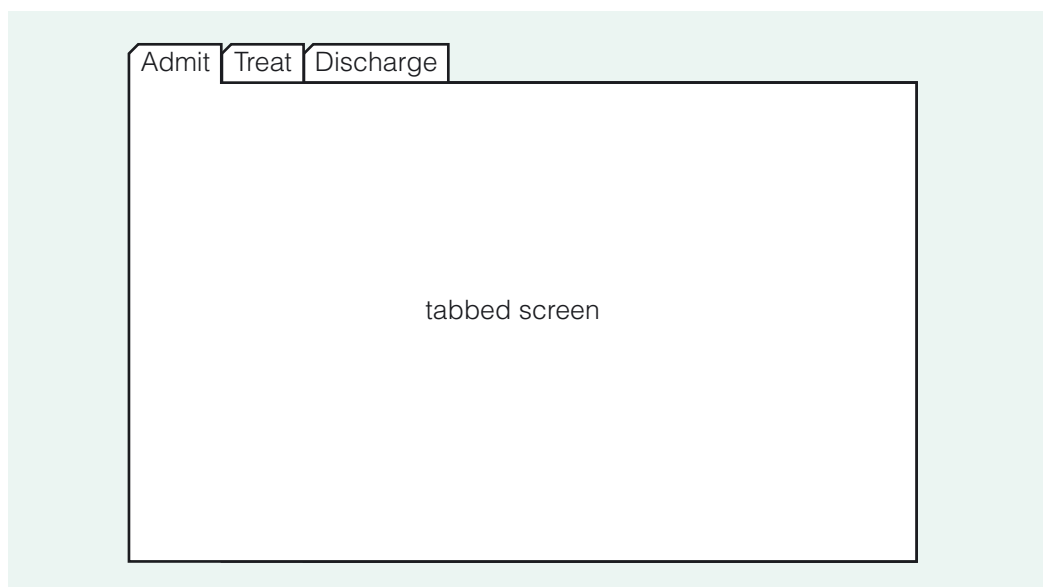


Figure 11 Some tabbed screens for the Hospital System

We will now create a design for two of the Hospital System use cases on the basis that each use case will have its own distinct screen for the user interactions involved. In a more complex system not every use case would correspond to a single user interface screen, but even then this can be a useful starting assumption: GUI designs for separate use cases may initially be created and assessed separately, before being combined.

5.3 A screen for *Admit Patient*

To begin, we will consider the *Admit Patient* use case, whose description is reproduced below from the requirements document.

The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.

The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen. If there is no appropriate ward with empty beds the administrator is informed of this fact.

If there is an appropriate ward with empty beds the system does the following two things.

- 1 It records:
 - ▶ the patient's name, sex and date of birth;
 - ▶ that the patient is under the care of the given team;
 - ▶ that the consultant that heads the team is responsible for the patient;
 - ▶ that the patient is on the ward.
- 2 It informs the administrator of the ward to which the patient has been admitted.

An initial prototype

Since the purpose of a user interface is to mediate information between the user and the system, we first examine this use case description with the aim of:

- ▶ exploring the **information flow** – the input (from the user) and output (from the system);
- ▶ using the information flow to select appropriate widgets.

It can be useful to set this information out in an **initial widget table**, such as Table 2. (Note that this particular table is incomplete – the remaining entries will be discussed shortly.)

Table 2 Incomplete initial widget table for the *Admit Patient* use case

Information	Input or output	Widget
Name	Input	
Sex	Input	Two radio buttons
Date of birth	Input	
Team that cares for patient	Input	
Message: 'No appropriate ward with empty beds' (in the case where there is no appropriate ward with empty beds)	Output	Text area
Ward to which patient has been admitted (in the case of an appropriate ward with empty beds)	Output	

SAQ 13

Why are two radio buttons appropriate widgets for entering the sex of a patient?

ANSWER.....

Two radio buttons are appropriate because there are two choices, male and female, precisely one of which must be selected.

Identifying input and output information can raise many queries about the form of the information, the answers to which will influence the initial choice of widgets. Issues for clarification in this case include the following.

- ▶ Should the patient's name be entered as a single item of information, or split into multiple items each entered separately (for example, first name, family name, middle initials)?
- ▶ Precisely what information about the ward should the user be given when there is an appropriate ward with an empty bed?

SAQ 14

Suggest other queries about information that might be raised in the drawing up of an initial widget table for *Admit Patient*.

ANSWER.....

There are no hard and fast answers to this; here are two queries that we noted.

- ▶ In what form should a date of birth be input? Using digits, e.g. 24/12/00, or in textual form, e.g. 24 December 2000?
- ▶ How should a team be identified?

These are the kinds of question for which answers would be sought from the client or through the activities of a user needs analysis, as noted in Subsection 3.2. For our purposes, we will assume that the answers are as follows.

- ▶ A patient's name will be entered as three separate items of information:
 - title;
 - first name;
 - surname.
- ▶ When there is an appropriate ward with an empty bed the user should be told the name of the ward.
- ▶ Dates of birth should be entered in the format dd/mm/yy, e.g. 24/12/00.
- ▶ Teams should be identified by team code.

On the basis of such additional information, widgets can be chosen and an initial prototype sketched.

SAQ 15

Given the additional information above, and assuming there are usually around 15 teams in the hospital, what widget(s) would you suggest be used to enable the required team information to be input?

ANSWER.....

A list widget of the team codes would be appropriate. Given the likely number of team codes, radio buttons, which potentially offer the functionality required, would be less suitable.

SAQ 15 allows us to highlight an important point, which is that choosing appropriate widgets tends to result in a GUI which automatically satisfies some of the design heuristics. Assessment of the appropriateness of a widget is often based on the factors that are at the heart of the heuristics. A list widget, for example, is appropriate for enabling information about a team to be provided to the system; it is appropriate because it allows the user to recognise and select a team code, rather than to memorise and enter it.

Possible widgets you should consider are described in Section 5 of Unit 11.

SAQ 16

To which of the ten design heuristics given in Subsection 4.1 above does this appropriate choice of a list widget most obviously relate?

ANSWER.....

This choice most closely relates to the fourth heuristic:

Minimise the users' memory load

Having made initial choices for widgets, Table 2 might be completed as shown in Table 3. There are of course other widgets that could be considered appropriate (in particular, there are more complex widgets that are beyond the scope of this course), but our subsequent work will be based on the choices in the table below.

Table 3 Initial widget table for the *Admit Patient* use case

Information	Input or output	Widget
Name (title, first name, surname)	Input	Three text fields, one for each of title, first name and surname
Sex	Input	Two radio buttons
Date of birth (dd/mm/yy)	Input	Text field
Team that cares for patient	Input	List of team codes
Message: 'No appropriate ward with empty beds' (in the case where there is no appropriate ward with empty beds)	Output	Text area
Ward to which patient has been admitted (in the case of an appropriate ward with empty beds)	Output	Text area

This table highlights an important point, noted earlier, which is that the development of the core system and the user interface proceed in parallel, and that as they go along each of these activities continually informs the other. In the case of the Hospital System the coordinating class has been equipped not only with a suitable coordinating method for admitting a patient (the method `admit(aName, aSex, aDate, aTeam)`), but also with all the methods the user interface requires to provide arguments to the coordinating method and retrieve attribute values from core objects. This was discussed in *Unit 7*, Section 5, where we anticipated what the requirements of the user interface would be. Here, for a list of team codes to be provided to the user, the core system must allow the user interface access to the code of each team. In *Unit 7*, anticipating this requirement of the user interface, we equipped:

- ▶ the coordinating class of the core system with a method `getTeams()`, which returns a collection of all the `Team` objects in the core system;
- ▶ each class in the core system with a getter method for each of its attributes – so the `Team` class has a getter method `getCode()`, which returns the code of a `Team` object.

In *Unit 13* you will see how the services provided by the core system are actually employed in implementing the GUI.

Based on the information gathered, together with an initial widget table such as Table 3, an initial screen can be sketched. Figure 12 shows one possible initial screen for *Admit Patient*. This is of course only one of many designs possible using the widgets in Table 3.

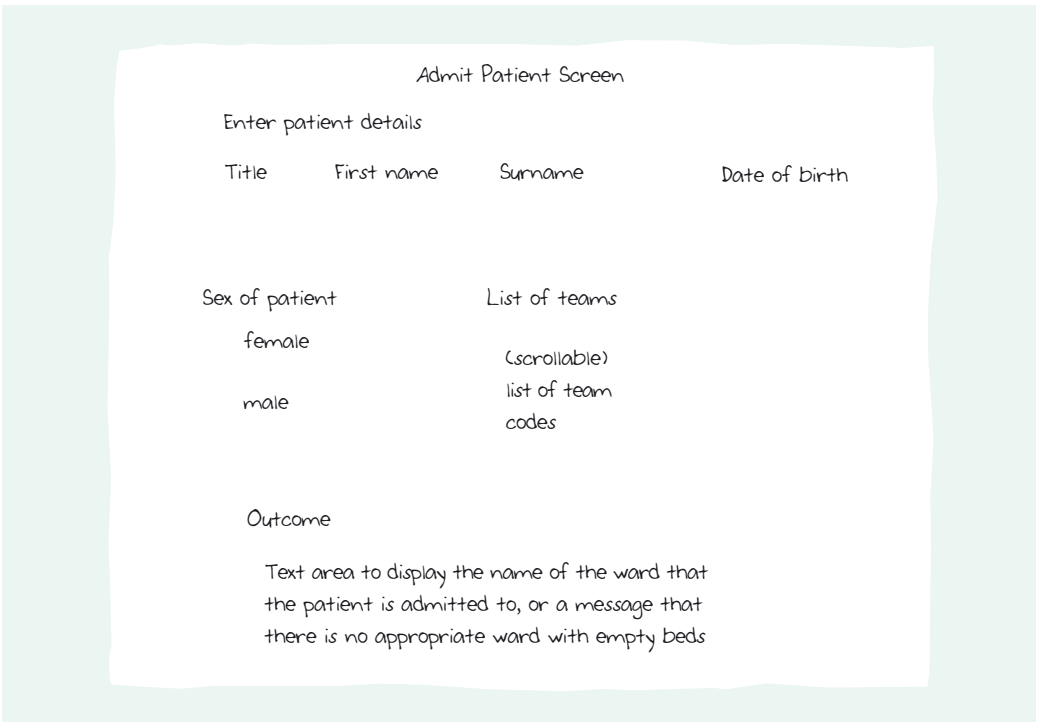


Figure 12 Initial prototype screen for *Admit Patient*

This design is far from perfect, and it will be improved in several ways. However even a very basic initial sketch such as this can be refined using the design heuristics to produce a satisfactory GUI.

Heuristic evaluation of the *Admit Patient* screen

As you know, heuristic evaluation can be carried out very simply by considering the heuristics in turn to see if any of them inspires an improvement to the GUI. For an example, consider the fourth design heuristic:

Minimise the users' memory load Avoid requiring users to memorise information, instead giving them opportunities to recognise, select, confirm or modify.

The prototype screen in Figure 12 currently gives no information about the form in which the date of birth should be entered: users would have to remember the required form and mistakes would be likely. On this basis a design rationale can be set out documenting a suggested improvement, as follows.

Design heuristic *Minimise the users' memory load*

Issue The user has to remember the required date format.

Suggested improvement Change the label on the **Date of birth** text field to **Date of birth (dd/mm/yy)**.

As another example consider the label at the top of the screen in Figure 12. This gives the title of the screen: **Admit Patient Screen**. In Western countries, users read from top to bottom so this label at the top is well placed for the user to find out quickly if they are on the right screen. However the text used partly reflects the designers' concern ('it's a screen') rather than the users' concern ('I want to admit a patient'). Hence the improvement in the following design rationale might be appropriate.

Perhaps to a hospital administrator 'screen' will first suggest something placed round a bed!

Design heuristic *Speak the users' language*

Issue The **Admit Patient Screen** label does not fully reflect the users' perspective.

Suggested improvement Change the label **Admit Patient Screen** to **Admit Patient**.

This change is relatively trivial in itself, but a series of small changes can measurably improve factors such as effectiveness, efficiency and satisfaction. Consequently such issues should be taken seriously.

Exercise 6

For each of the design heuristics below give a design rationale for a corresponding improvement to the initial prototype screen for *Admit Patient* (Figure 12).

- (a) *Allow for undo-ability and commitment*
- (b) *Use simple and minimal dialogue*
- (c) *Include clearly marked exits*

Discussion.....

Here are some possibilities.

- (a) **Design heuristic** *Allow for undo-ability and commitment*

Issue The screen allows unlimited altering of the input, but there is no explicit way of making clear that the currently entered information should actually be input to the system.

Suggested improvement Add an **Admit Patient** button.

- (b) **Design heuristic** *Use simple and minimal dialogue*

Issue The text on the screen does not give the necessary instructions to the user. **Enter patient details** is an instruction, but **Sex of patient** and **List of teams** are not, and a novice user might not realise that they are required to input information in these places.

Suggested improvement Provide a set of instructions: change **Enter patient details** to **1. Enter patient details**; change **Sex of patient** to **2. Select sex of patient** and change **List of teams** to **3. Select team that cares for the patient**; add a label **4. Click Admit Patient button**.

- (c) **Design heuristic** *Include clearly marked exits*

Issue The GUI so far lacks any way to exit the system.

Suggested improvement Add an **Exit** button.

A second prototype

On the basis of the changes suggested by a heuristic evaluation such as the above, the initial prototype GUI can be improved. In a real project, as you learnt in Section 3, usability testing of the prototype would also influence the improved design.

SAQ 17

What three characteristics of a user's interaction with the prototype might be measured in usability testing?

ANSWER.....

Section 3 mentioned:

- 1 effectiveness;
- 2 efficiency;
- 3 satisfaction.

Since you are not in a position to evaluate users' interaction with the prototype, we will concentrate solely on the design heuristics. On the basis of heuristic evaluation, you might arrive at something like the screen sketched in Figure 13.

Figure 13 Second prototype screen for *Admit Patient*

Although we will go no further than this second prototype, it is important to recognise that in reality several further prototypes may be created. These might address issues such as the following.

- ▶ What should happen if invalid information (an invalid date, for example) is entered? If there is to be an error message, should this be provided immediately the date is entered or only when the **Admit Patient** button is clicked?
- ▶ If the admission of a patient is successful, a message appears in the **Outcome** area, but what happens to the input fields? Should they still contain the information that the user entered or be blanked out in preparation for the next admission?
- ▶ What should happen to the input fields if the admission is unsuccessful because there are no appropriate wards with empty beds?

Some of these issues will, however, be considered in Unit 13.

5.4 A screen for *Treat Patient*

The other Hospital System use case for which a screen will be designed in this unit is *Treat Patient*, whose description from the requirements document is as follows.

The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

To begin, an outline initial widget table for this use case might look as follows.

Table 4 Incomplete initial widget table for the *Treat Patient* use case

Information	Input or output	Widget
Patient	Input	
Doctor	Input	

At this point, issues are raised about the form in which the information should be input.

SAQ 18

What questions about information would you, as a designer, want to ask?

ANSWER.....

You would probably want to find answers to the following, at least.

- ▶ How should the patient be identified?
- ▶ How should the doctor be identified?

Exercise 7

Assume that the answers to the questions raised in SAQ 18 are that both patients and doctors should be identified by name.

Based on the use case description, draw up a widget table for *Treat Patient*.

Discussion.....

Your table is likely to look something like Table 5. As with the *Admit Patient* use case, we have decided that a list would in each case be an appropriate widget, to minimise the memory load on the user.

Table 5 Initial widget table for the *Treat Patient* use case

Information	Input or output	Widget
Patient	Input	List of patient names
Doctor	Input	List of doctor names

We are taking a very simple approach to the information here. A more realistic solution, for example, could require doctors to have a staff number and patients to have some kind of identifying number. Such decisions would have consequences for the core system.

Exercise 8

On the basis of Table 5, sketch an initial prototype screen for *Treat Patient*.

Discussion.....

Figure 14 shows a possible prototype screen for *Treat Patient*. Your sketch will probably not look quite the same, but it should show the same widgets (i.e. as in Table 5) and have similar labelling.

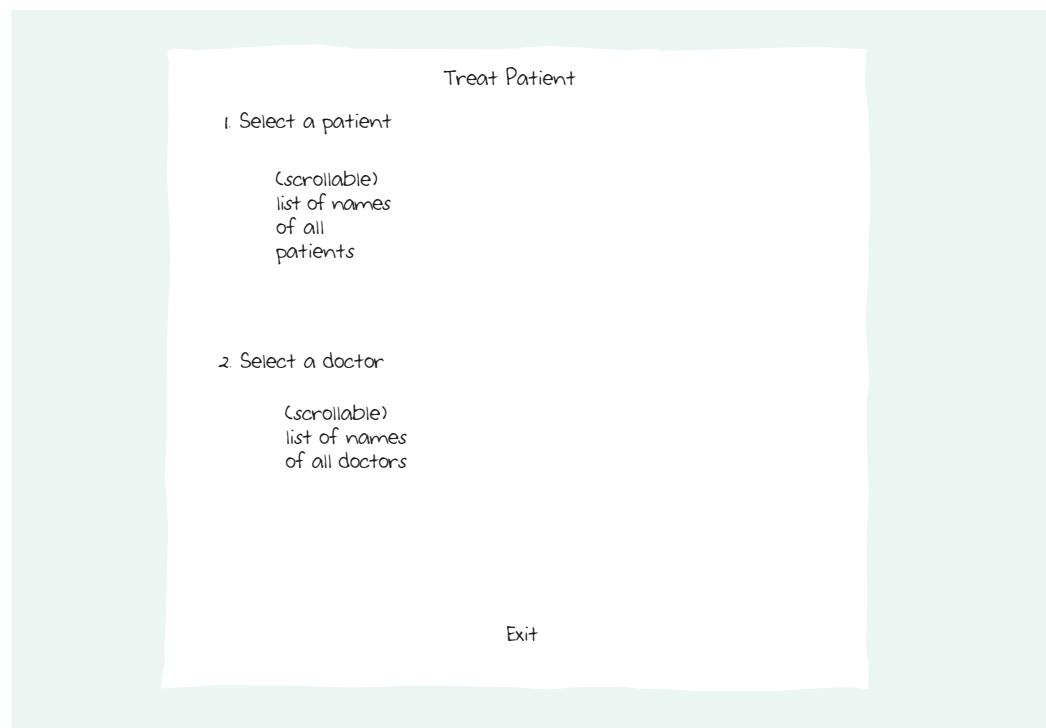


Figure 14 An initial prototype screen for *Treat Patient*

Exercise 9

For each of the design heuristics below, give a design rationale for a corresponding improvement to the initial prototype screen for *Treat Patient* (Figure 14).

(a) *Minimise the users' memory load*

For this part, recall from the description of the *Treat Patient* use case that the doctor must be in the team that cares for the patient.

(b) *Provide feedback*

(c) *Allow for undo-ability and commitment*

(d) *Prevent errors*

For this part, concentrate on how different patients with the same or similar names might be identified.

Discussion.....

Here are some possibilities.

(a) **Design heuristic** *Minimise the users' memory load*

Issue The user is required to remember which doctors are in the team that cares for the patient.

Suggested improvement Have the list of doctor names include only the names of the doctors in the team that cares for the selected patient.

Note that this user interface requirement was anticipated when designing the core system. The coordinating class, with whose coordinating object the user interface will communicate, was equipped with methods `getTeam(aPatient)` and `getDoctors(aTeam)` to enable the user interface to retrieve precisely the information it needs to list the names of only the doctors in the right team.

(b) **Design heuristic** *Provide feedback*

Issue Since the system does something in response to user input, feedback should be provided.

Suggested improvement Add a text area for this feedback.

In general, there should be feedback for all use cases. If the state of the system is changed, the user wants to know this has happened successfully; for a use case that only finds out information, this information is the feedback.

(c) **Design heuristic** *Allow for undo-ability and commitment*

Issue As with the *Admit Patient* use case the difference between exploring options and commitment is not clear.

Suggested improvement Add a **Treat Patient** button to enable the user to confirm when they are happy with the currently selected options.

(d) **Design heuristic** *Prevent errors*

Issue If there were two patients with the same or similar names, the wrong patient might be identified and an incorrect treatment recorded.

Suggested improvement In a text field, display the date of birth of the currently selected patient, to help in selecting the right patient.

Exercise 10

Sketch an improved version of the screen in Figure 14 on the basis of the heuristic evaluation performed above.

Discussion.....

Figure 15 shows a possible second prototype screen.

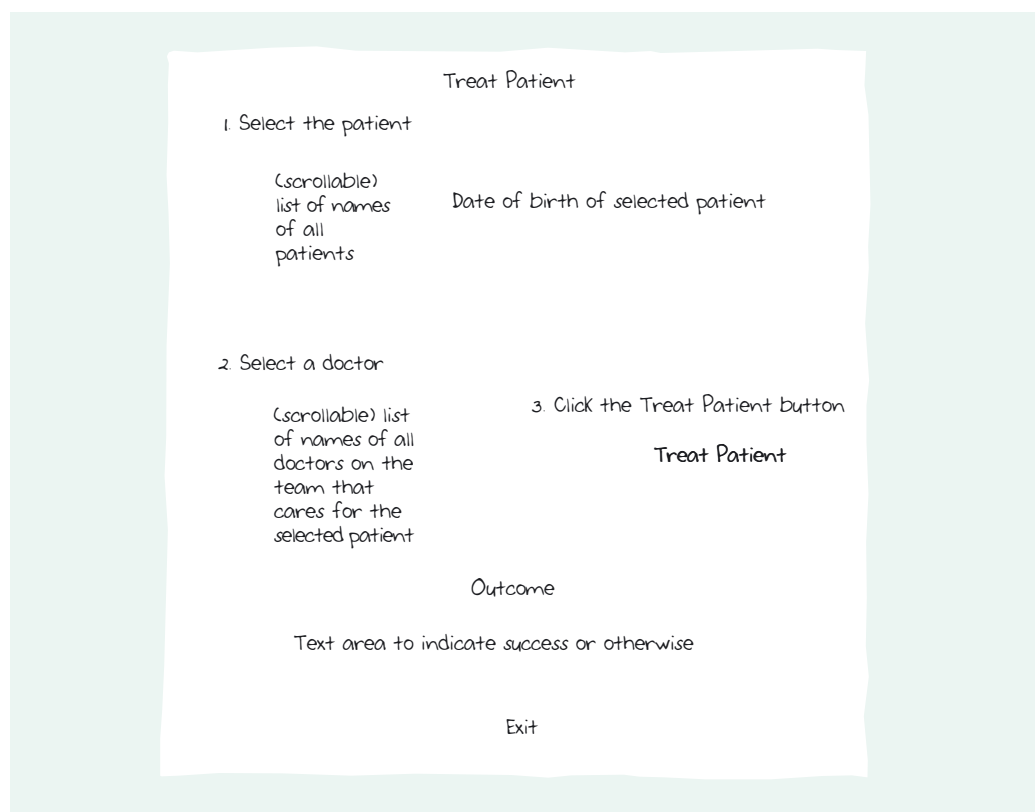


Figure 15 A second prototype screen for *Treat Patient*

In this section you participated in the creation of an initial prototype screen for two of the Hospital System use cases. In each case you then performed a heuristic evaluation and used the results as a basis for creating a second prototype.

6

Summary

This unit began by considering some landmarks in GUI design that illustrate important HCI issues. We explored various connections between HCI and object-oriented ideas, and we outlined the seminal design ideas behind the Xerox Star.

The concepts of user-centred design and usability were revisited, and you saw the importance of gathering information about the users, their tasks and their situation. The central importance of prototyping and usability testing was considered.

Stemming from early design ideas, such as those used in developing the Xerox Star, the use of different kinds of user interface design guidance such as principles and heuristics were discussed. Heuristic evaluation was introduced to give you a simple, practical framework for improving prototypes.

Finally, in the context of creating GUI designs for the Hospital System, you learnt about a simple approach to creating initial prototypes and improving them through the process of heuristic evaluation.

In *Unit 13* you will use the designs arrived at in this unit as a basis for implementing parts of a GUI for the Hospital System.

LEARNING OUTCOMES

After studying this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ explain the connection between GUI interfaces and object-oriented ideas;
- ▶ explain the design ideas at the heart of the Xerox Star;
- ▶ explain what can be involved in gathering information for a user needs analysis;
- ▶ use the ISO definition of usability as the basis for forming appropriate questions as part of a user needs analysis;
- ▶ explain what can be involved in usability testing;
- ▶ use a design heuristic to identify a fault with a user interface, and suggest an improvement;
- ▶ identify information flows, queries about information and suitable widgets for an initial prototype interface;
- ▶ design an initial prototype interface on the basis of an initial widget table;
- ▶ design an improved prototype interface on the basis of heuristic evaluation.

Glossary

animate agent An independently acting cause of change in the human environment.

design for all See universal access.

design heuristic A piece of relatively concrete advice for designing a user interface: a rule of thumb that may not always be applicable, but is frequently useful. (Also referred to as a *usability heuristic* or simply a *heuristic*.)

design rationale Documentation of a designer's suggested improvement to a user interface, in the context of a particular design heuristic.

effectiveness (in the context of a user interface) How well the user interface enables the user to get the right thing done.

efficiency (in the context of a user interface) How much of a resource has to be expended for a user to get a job done via the user interface.

heuristic See design heuristic.

heuristic evaluation An assessment of a user interface with reference to particular design heuristics.

information flow The input from the user and the output from the system.

initial widget table A table setting out the information flow and the widgets to be used in the initial prototype screen for a particular use case.

NASA TLX (Task Load Index) tool A tool that measures a user's impression of their workload when carrying out some task, based on a multidimensional rating procedure that calculates an overall workload score based on a weighted average of ratings on six subscales: frustration, mental demands, physical demands, temporal demands, own performance and effort.

Reasoning by class An innate form of human reasoning, derived from a form of reasoning about animals, which can be described as follows.

If we observe that just one animal of a particular unfamiliar type has a certain kind of behaviour (e.g. bites, flies, lays eggs), then it is a fairly good assumption that any other animal of that type will do the same.

Reasoning by inheritance An innate form of human reasoning, derived from a form of reasoning about animals, which can be described as follows.

If we are given the single piece of information that one type of animal (animal A) is a special kind of another animal (animal B) then any time we encounter animal A, we will know that it can do everything that animal Bs do, and probably some extra things.

Sketchpad The first computer system to make a substantial contribution to the discipline of HCI, implemented by Ivan Sutherland in 1962 as part of his PhD thesis.

storyboard A diagrammatic representation of a user interface, which may show the navigation through screens.

tabbed screen One of a set of screens, each with a tab at the top. To move from one screen to another, the tab of the desired screen is clicked on.

think-aloud A simple usability testing technique which involves the user speaking aloud what they are thinking when using the user interface to perform tasks.

universal access An approach to development whose key idea is that effort should be expended to find simple ways in which small changes to a system could make it more widely accessible (also referred to as *design for all*).

usability Refers to how easily and quickly a product enables users to achieve their goals, how quickly users learn to use the product's interface and what the users' attitude is towards the interface. The ISO 9241 (Part 11) definition of usability is:

The extent to which a product can be used by specified users to achieve specified goals [tasks] with effectiveness, efficiency, and satisfaction in a specified context of use.

usability heuristic See design heuristic.

usability testing Assessing the usability of an interface.

user-centred approach An approach to developing software which focuses on what users find easy and what they find difficult.

user interface designer A software developer whose particular responsibility is the user interface.

user needs analysis document A record of the results of user needs analysis, containing information about users, tasks and situations, and giving user interface designers a concrete basis on which to work.

user satisfaction (in the context of a user interface) How satisfied the user is in interacting with the system via the user interface.

Xerox Star A landmark computer system in HCI history, which laid the foundations of modern GUI interfaces and was associated with seminal user interface design ideas.

References

Ingalls, D. (1978) 'The Smalltalk-76 Programming System Design and Implementation', in *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages*, ACM. Available online at: <http://users.ipa.net/~dwichth/smalltalk/St76/Smalltalk76ProgrammingSystem.html> (Accessed 21 March 2007).

Norman, D. A. (2002) *The Design of Everyday Things*, Basic Books Inc., New York, USA.

Acknowledgements

Grateful acknowledgement is made to the following sources.

Figure 2: Reproduced by permission of Alan Kay.

Figure 3: The Digibarn Computer Museum/Dave Curbow (www.digibarn.com).

Figure 6: Norman, D. and Draper, S. (eds) (1986) *User-Centred System Design*, originally published by Lawrence Erlbaum Associates Inc., by permission of Taylor and Francis Inc.

Figure 7: Norman, D. and Draper, S. (eds) (1986) *User-Centred System Design*, originally published by Lawrence Erlbaum Associates Inc., by permission of Taylor and Francis Inc.

Figure 8: Nomadic Research Labs Image Library.

Every effort has been made to contact copyright holders. If any have been inadvertently overlooked the publishers will be pleased to make the necessary arrangements at the first opportunity.

Index

A

aesthetics 18
affordance 29
animate agent 8
Apple
 Lisa 11
 Macintosh 11

C

cognitive and evolutionary psychology 8
collaboration 21
commitment 30
computer games 11
computer games console
 Microsoft Xbox 11
 Sony PlayStation 11
concept (user-oriented) 30
consistency 13, 30, 34
coordinating class 39
core system 6, 25, 39

D

design
 for all 19
 heuristic 30, 35
 principle 13, 25, 29
 rationale 32
desktop 13
developer 6
dialogue (simple and minimal) 30
dialogue control 21
direct manipulation 13, 17
documentation 23
drawing device (for child)
 Etchasketch 20–21
 Skedoodle 20

E

effectiveness 27
efficiency 27
error message 30

error prevention 30, 32
Etchasketch 20–21
exits (clear marking of) 30

F

feedback 12, 29–30

G

graphical user interface (GUI/GUI interface) 6
grouping and ordering 30, 34
GUI widget 11

H

HCI (human–computer interaction) 7
heuristic 30
heuristic evaluation 31

I

indirect users 19
inference (skill of) 8
information flow 37
inheritance 16
initial widget table 37
interaction 12
interactive products 6
ISO (International Organization for Standardization) 18

L

language (user-oriented) 30, 34
LaTeX 14–15
Lisa 11

M

Macintosh 11
memory load (of users) 30, 34
mental model 13, 17
metaphor 13
 desktop 13
 physical manipulation 13
 physical space 13, 17

Microsoft Windows 11

Microsoft Xbox 11

N

NASA TLX (Task Load Index)
 tool 27
Nielsen, Jakob 30

O

object 17
object-oriented technology and GUI interfaces 11, 16
operating systems 11, 19
ordering and grouping 30, 34

P

paperless office 9
polymorphism 16
prototype 25

Q

questionnaires 23

R

radar 8
reasoning
 by class 9
 by inheritance 9

S

screen 31
seeing and pointing (rather than remembering and typing) 14
Simula 10
situational analysis 24
Skedoodle 20
Sketchpad 7
Smalltalk 10
socio-technical system 21
Sony PlayStation 11
storyboard 31

T

tabbed screen 36

task

- allocation 21
- analysis 24
- success rates 27

terminology (user-oriented) 30, 34

think-aloud 27

U

undo-ability 30

universal access 19

universal commands 15–16

usability 29

- heuristic 30
- testing 26

user

- characterisation 24
- needs analysis 18
- needs analysis document 24
- observation 24
- satisfaction 27
- workload 27

user interface designer 6

user-centred

- approach 12, 25
- design 18, 35

V

visibility 29

Visicalc (first spreadsheet application) 12

W

WYSIWYG (what you see is what you get) 14

X

Xerox PARC 9

Xerox Star 9, 13, 16, 29